

Tarea técnica — Palvi

Hola, gracias por avanzar a esta etapa. Esta tarea está pensada para ver cómo piensas un problema, no cuántos features puedas meter. Apunta a entregar algo acotado y bien fundamentado.

El problema

Vas a recibir un único archivo `metrics.json` con métricas diarias de una empresa B2B SaaS: tráfico, leads, deals, tiempos de respuesta, tickets de soporte, etc.

Construye una **aplicación web que sirva como reporte ejecutivo** de esas métricas.

Contexto de uso: el **Jefe de Ventas** abre esta página en la mañana. Tiene 5 minutos antes de su primera reunión. Su trabajo es **aumentar las ventas y mejorar la atención al cliente**, y necesita salir sabiendo dónde poner foco hoy.

Importante: `metrics.json` trae 4 **datasets** (A, B, C, D) — misma estructura, comportamiento diferente. Tu app tiene que dejar al usuario navegar entre ellos y **responder correctamente a cada uno** — no solo al primero.

No hay una solución única. Tus decisiones importan más que la cantidad de cosas que metas.

El dataset

`metrics.json` tiene esta estructura — un objeto cuyas claves de primer nivel son los **datasets** (A, B, C, D), cada uno con su propia metadata y serie diaria:

```
{
  "A": {
    "metadata": {
      "start_date": "2025-04-26",
      "end_date": "2026-04-25",
      "days": 365,
      "metrics": [
        {
          "key": "traffic",
          "label": "Daily visits",
          "unit": "visits",
          "direction": "higher_is_better",
          "description": "Unique visits to the public marketing site."
        }
      ]
    },
    // ... resto de las métricas
  },
  "days": [
    {
      "date": "2025-04-26",
      "metrics": {
        "traffic": 1834,
        "leads_created": 12,
        "avg_response_time_min": 31.2
      }
    },
    // ... resto de las métricas
  ]
}
```

```

    // ... 365 días
  ]
},
"B": { /* misma forma */ },
"C": { /* misma forma */ },
"D": { /* misma forma */ }
}

```

Las 4 versiones comparten la misma lista de métricas en `metadata.metrics`. Cambia el comportamiento subyacente de los datos.

El campo `direction` indica si subir es bueno (`higher_is_better`) o malo (`lower_is_better`) — eso te lo damos para que no tengas que adivinar el dominio.

Algunas métricas pueden venir como `null` en días puntuales (por ejemplo, `avg_response_time_min` cuando no hubo leads ese día). Considéralo.

Glosario

No esperamos que sepas vender. Estos son los términos que aparecen en `metrics.json`:

- **Lead** — persona o empresa que mostró interés (llenó un formulario, agendó demo). `leads_created` cuenta los nuevos del día.
- **Lead calificado** — lead que ventas evaluó y considera prospecto real (fit, presupuesto, timing). `leads_qualified`.
- **Deal** — oportunidad de venta abierta sobre un lead calificado. `deals_created` cuenta nuevas en el día.
- **Deal ganado / perdido** — `deals_won` y `deals_lost` cuentan los deals que cerraron hoy con cada resultado.
- **Tiempo de respuesta** (`avg_response_time_min`) — minutos desde que llega un lead hasta el primer contacto del equipo de ventas. En B2B, respuestas lentas tumban la tasa de conversión.
- **Deal cycle** (`avg_deal_cycle_days`) — días entre apertura y cierre. Promedio sobre los deals que cerraron ese día.
- **Stale deal** (`stale_deals`) — deal abierto desde hace **más de 60 días** sin cerrar. Conteo al final del día.
- **Tasa de cierre / win rate** — fracción de deals cerrados que se ganaron, sobre una ventana: $\text{sum}(\text{deals_won}) / \text{sum}(\text{deals_won} + \text{deals_lost})$. Métrica de período (qué cerró esta semana), no de cohorte (qué pasó con los deals abiertos en marzo).
- **Funnel / embudo** — tráfico -> leads -> leads calificados -> deals -> deals ganados. Cada paso tiene su tasa de conversión; un cuello en uno se nota aguas abajo.
- **Tickets de soporte** — `support_tickets_opened` cuenta los abiertos hoy; `support_avg_resolution_hours` es el promedio de horas para resolver los abiertos hoy.

Stack

React + TypeScript. Lo demás lo decides tú: librería de UI, charting, estado, build tool. Cada decisión que tomes nos interesa.

Sobre el uso de IA

Esperamos que uses IA (Cursor, Claude Code, Copilot, lo que prefieras). Así trabajamos nosotros. No es trampa.

Lo que evaluamos no es si escribiste cada línea a mano, sino cómo descompusiste el problema, qué abstracciones elegiste y qué decisiones tomaste tú versus la IA.

Entregable

1. Repo en GitHub

Público o invitándonos como colaboradores. Instrucciones claras para correrlo localmente.

2. README corto

1 página máximo. Solo dos secciones: - **Decisiones técnicas:** qué elegiste y por qué. - **Segunda iteración:** qué dejarías para después y por qué.

3. Video de máximo 3 minutos

Loom, Tella, YouTube unlisted, Google Drive, o lo que prefieras. **Sin intro, sin edición, sin pulir** — graba una toma directa. Estructura:

1. **Demo (60s):** Cambia entre dos datasets (A y B, o cualquier par). Muéstranos que la app responde diferente entre uno y otro.
2. **Walk-through del código (90s):** Abre tu editor. Muéstranos el archivo o la función donde decides cómo presentar cada métrica al usuario. Explica brevemente por qué la implementaste así.
3. **Lo que dejaste fuera (30s):** Una cosa que sabes que falta o que harías distinto, y por qué.

Si te pasas de 3 minutos no vamos a alcanzar a verlo completo.

Tiempo y plazo

- **Tiempo estimado:** 3 horas. Si te toma más de 4, entrega lo que tengas.
- **Plazo:** 5 días corridos desde que recibes esta tarea.

Tip: probablemente vale más tiempo pensando que tipeando.

Cualquier duda

Responde este correo. Si la duda es sobre qué construir exactamente, intenta responderla tú mismo y déjala documentada en el README — esa decisión también nos interesa.

Éxito, Equipo Palvi